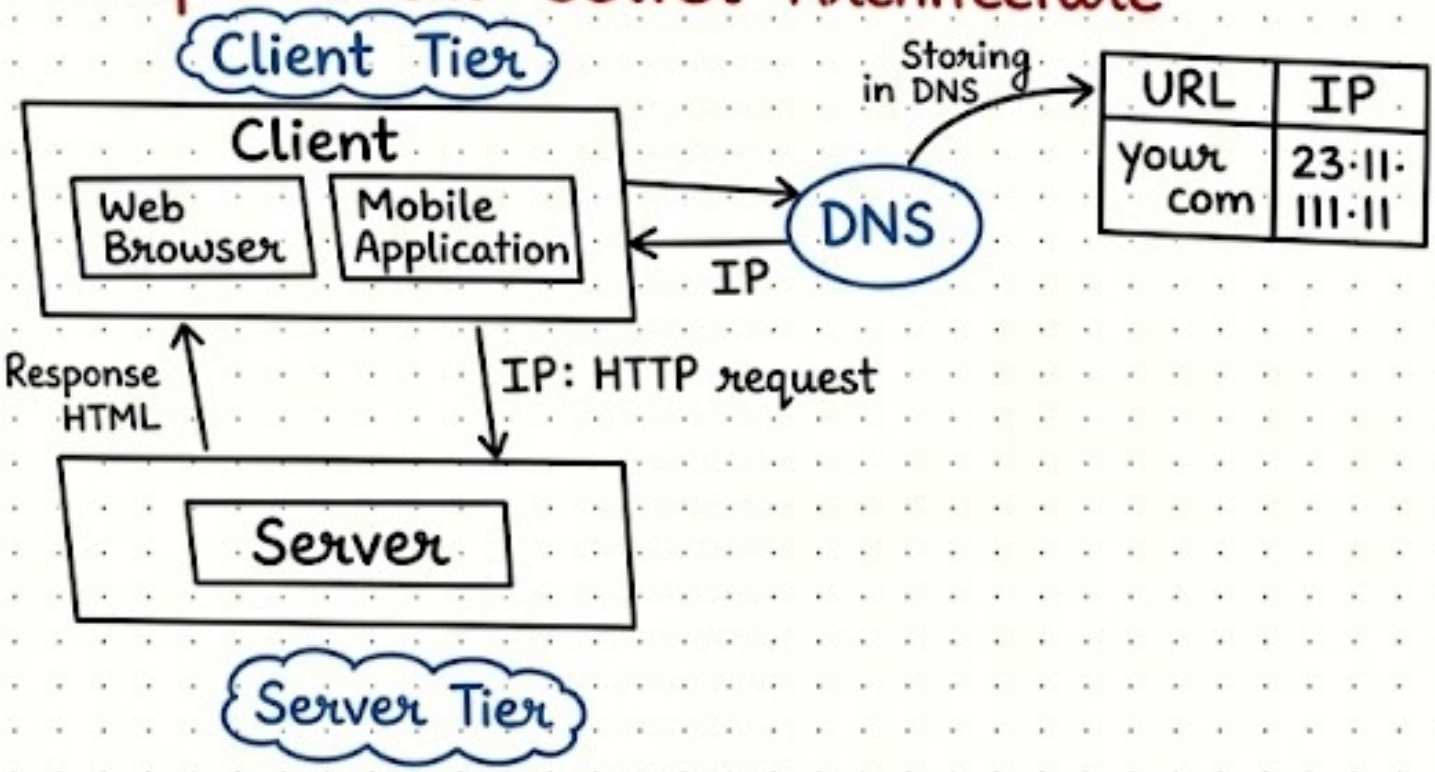
Lecture 2
Scaling from 0 to million users

- **Point to remember: in HLD**
  - ↳ Anything that we will do in HLD is to make our application scale better.

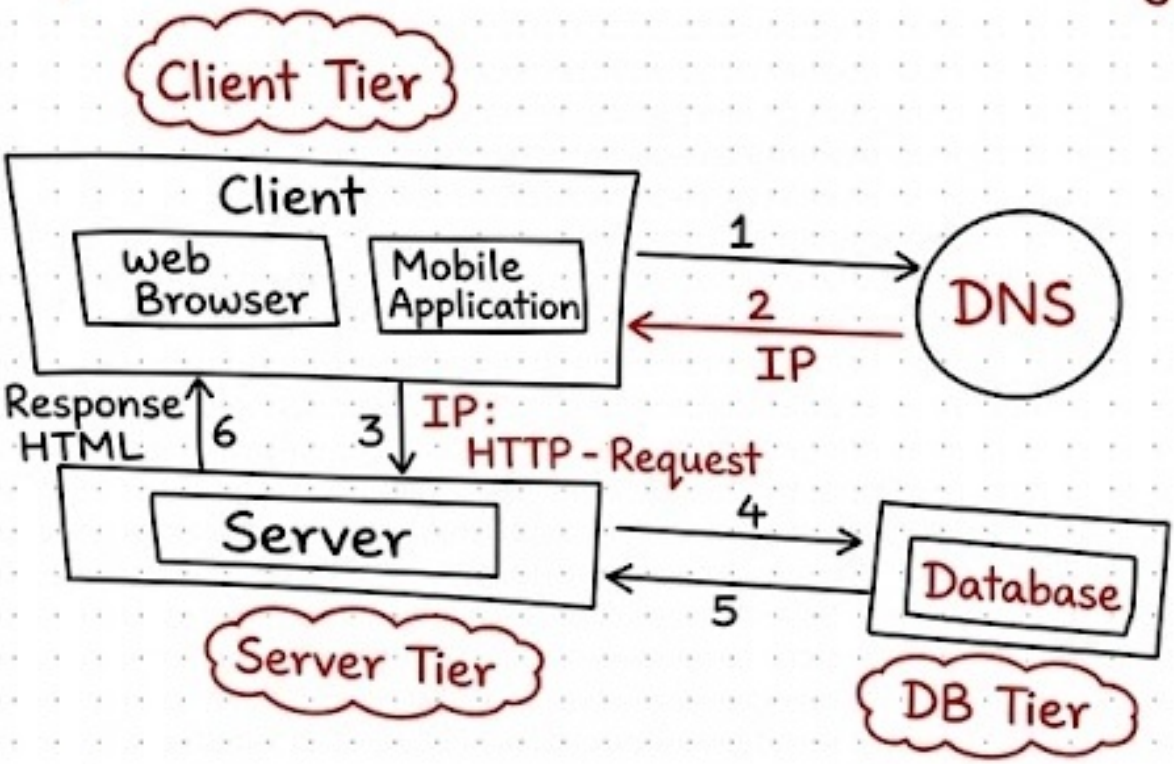
→ **Simple Client-Server Architecture**



Problems

↳ It can't be scaled much as our database is integrated with web server.

Improved version - Introduced DB layer



Discussion on DB

Which DB to use ?

Relational

1) MySQL
2) PostgreSQL
3) Oracle SQL

Non - Relational

① key - value store
↳ Amazon Dynamo DB

② Document based store
↳ MongoDB

③ Column based store
↳ Cassandra

④ Graph based store
↳ GraphQL

→ Store relationship data
Ex: insta ⇒ following each other (bidirectional graph).

Parameters to decide which db is great?

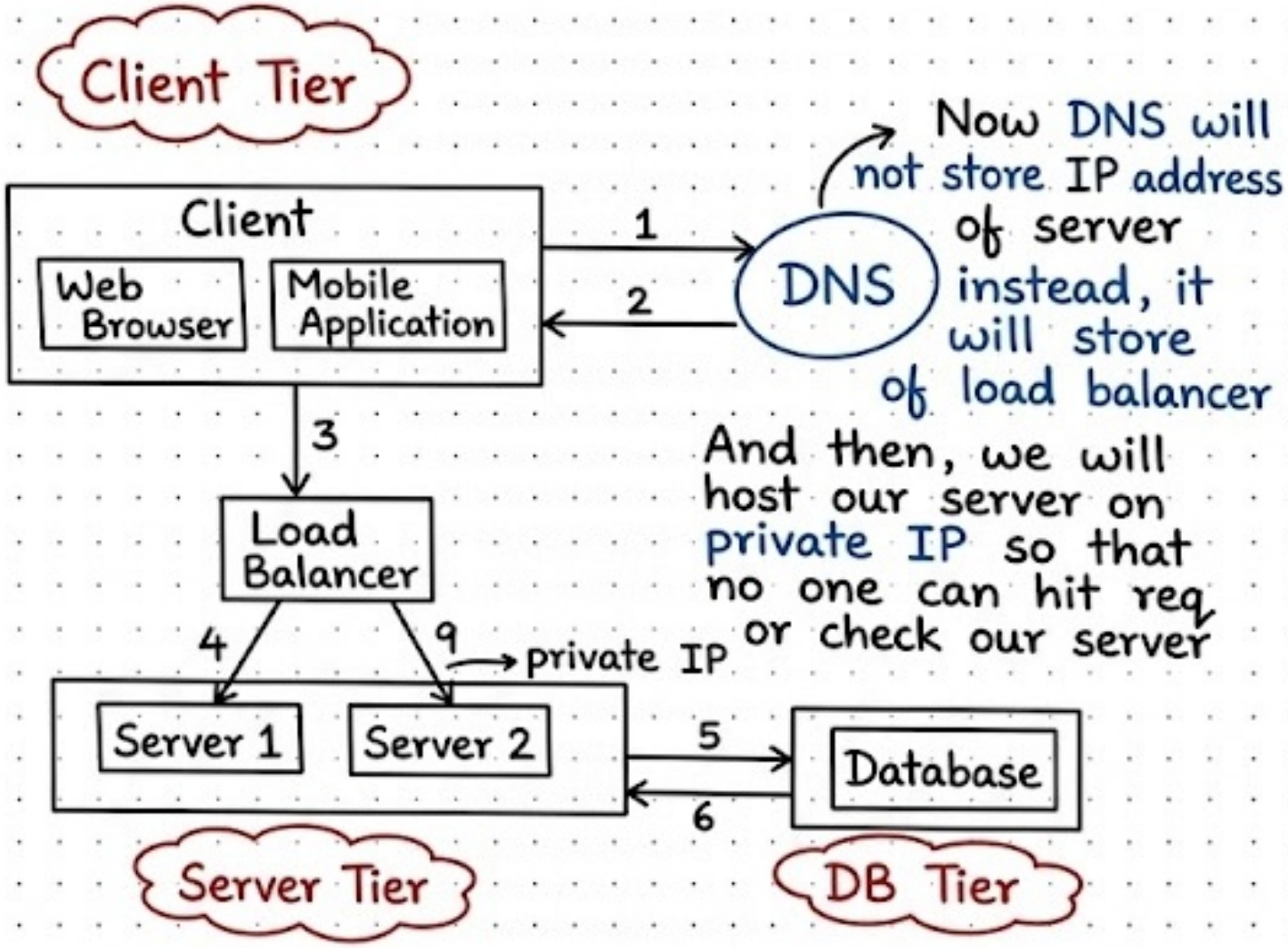
① Can we perform query on our db ⇒ yes ⇒ relational

② Faster execution of request / less latency ⇒ non-relational

③ But data is not structured ⇒ non-relational

Problem: 1) It can only handle few thousands users ⇒ we need to scale up but vertical scaling will lead to SPOF (single point of failure) ∴ we need horizontal scaling.

- Improved version: Multiple server + load Balancer
  1) We created multiple servers to handle requests.
  2) But how will DNS / we decide when to hit request on which server.
  3) To solve this we will introduce Load Balancer ∴ Balances the load on servers and redirect requests to server less busy / idle.



- Problem :-

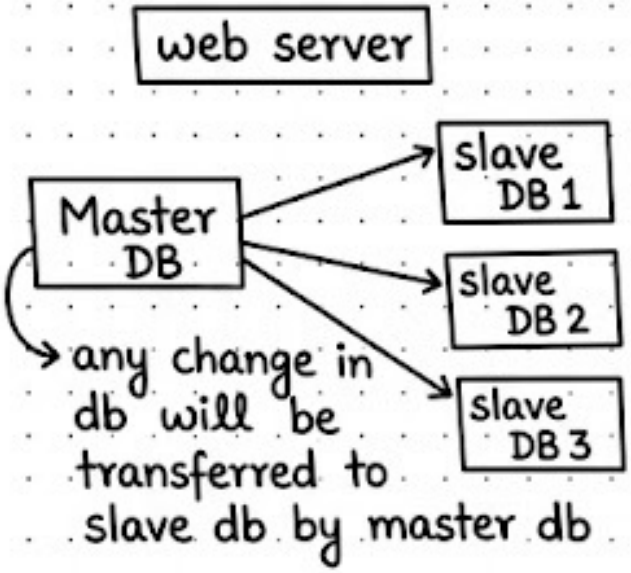
1) Now we have scaled servers

2) But our database is also a SPOF → we need to improve it to avoid physical damage on db and have data properly integrated

☆ Improved Version : DB Scaling

↳ To achieve this we will use Master slave architecture

Master-Slave Architecture



Queries that can be performed

- Read (Select): slave db
- write
- Insert
- Update
- Delete

} Master db

→ after update it will update it's own query

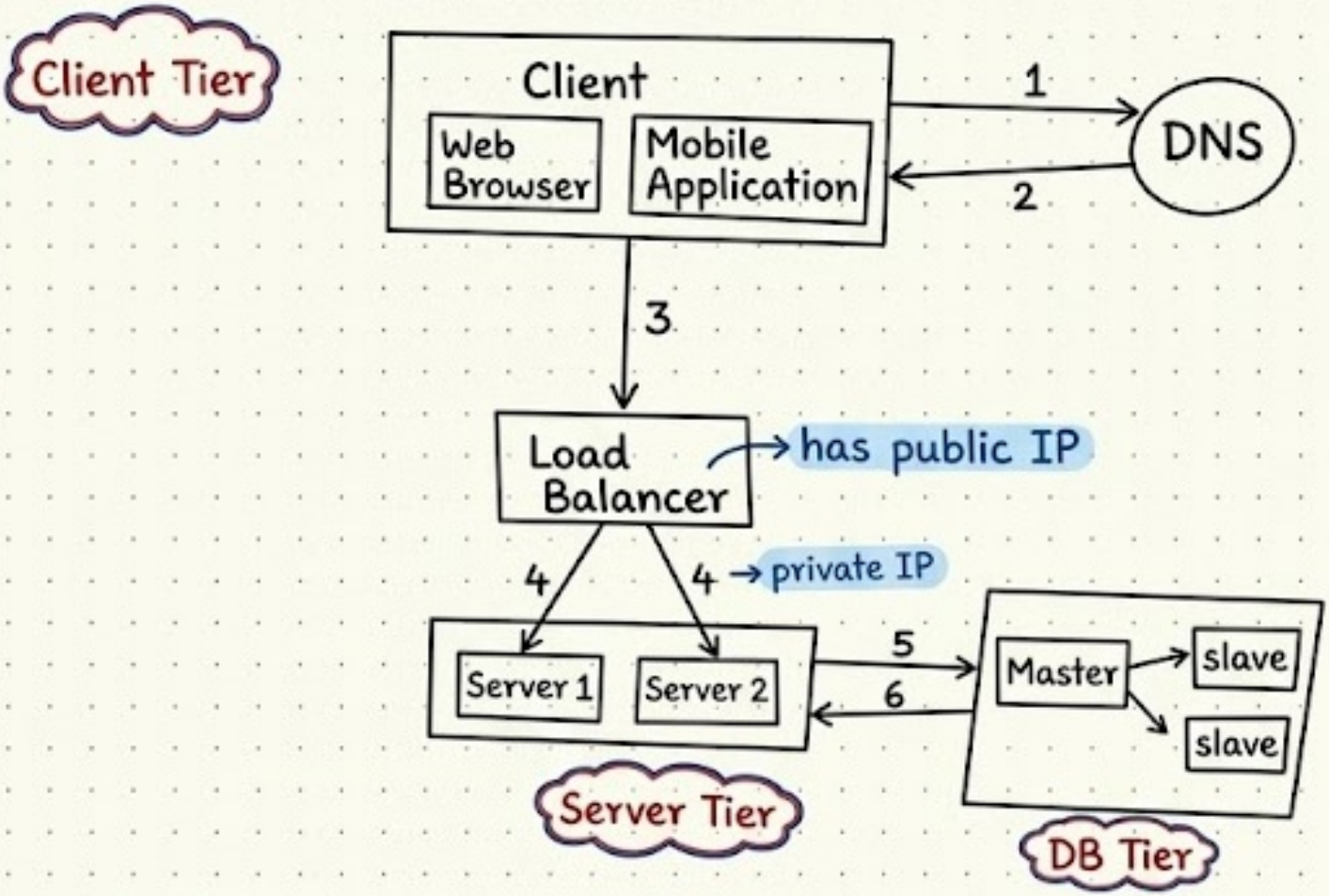
- Which slave server will give data
  - ↳ we will use load balancer to decide it.
  - → Later on discussion in detail.

- What problems are solved ?

1) If any slave db is crashed we will have option and data store in master db

2) what if master db crashed ?
   - ↳ In that case, we will make any of the slave db as master, till old master db is up or new master db is created
   - ↓
   - Which slave ? → multiple algorithms are used to decide it, most common is voting algorithm.

3) what if we made change in db like added insta post & before data is transferred to slave db, master db crashes ?
   - ↓
   - if master crashes before sending data to slave, the data can be lost.
   - To prevent this, we make sure data is saved in multiple places before confirming the write. ↓ (Replicas)

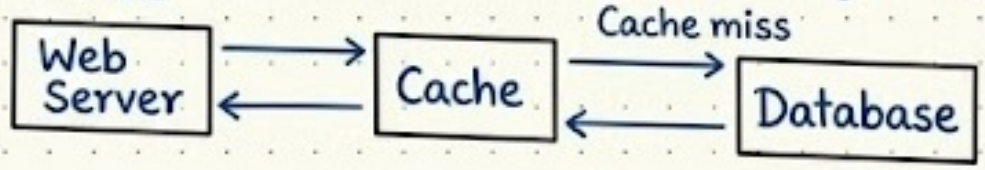


Que.] why do we need server code or language like Java/python in our application when HTML+CSS+JS are enough to handle any logic or processing, inshort: "why server side code?"

Ans ⇒ If we did not introduced server side code then our database will be exposed and it will lead to data loss, wrong data injection, security issues and no authentication.

★ Improved Version: Cache

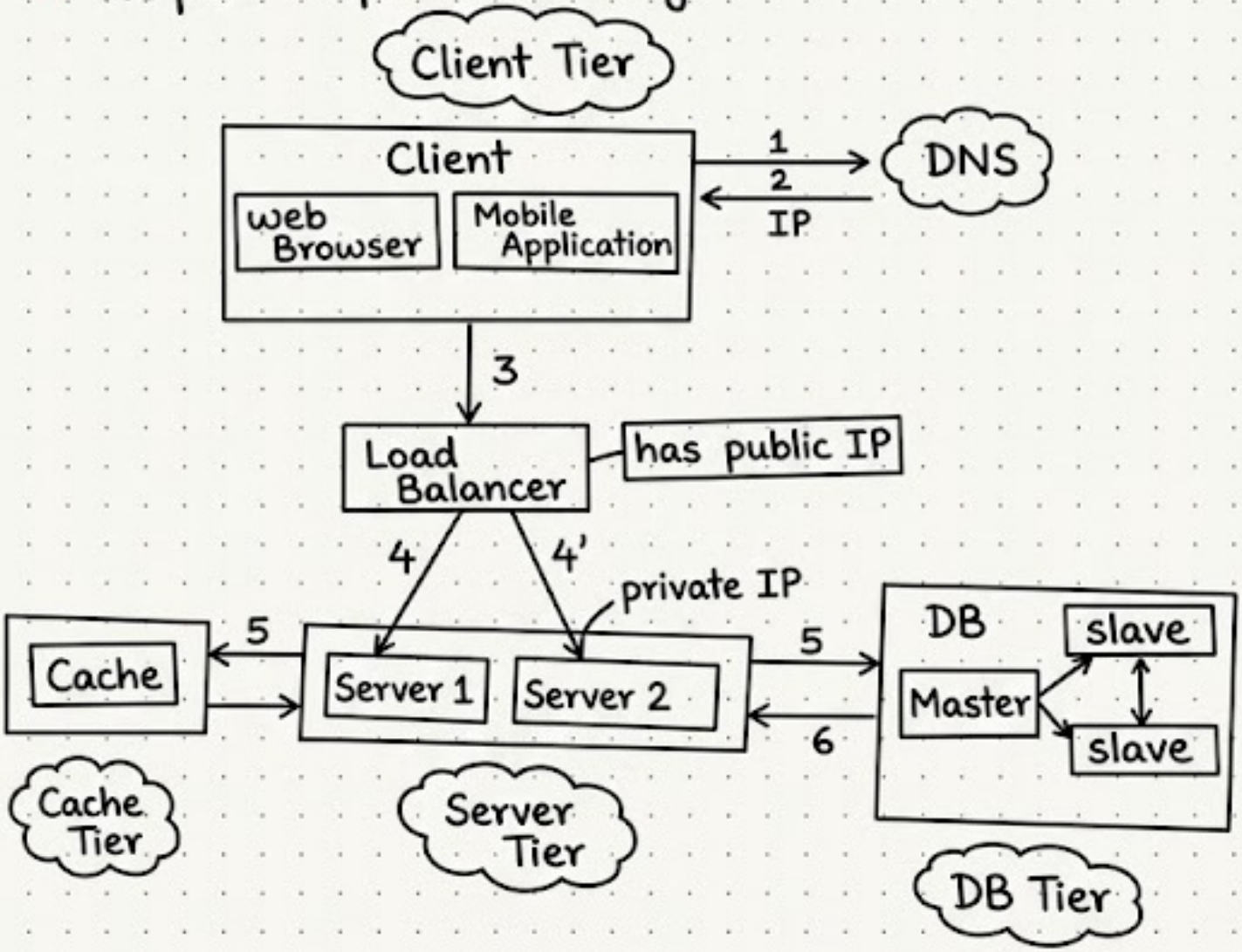
↳ Cache helps in reducing query request on db



- Cache has many algorithms and internal implementation

Cache Eviction Algorithm - decides which and when data is removed when cache is removed.

- Cache - It is faster than DB, any operation like read and write are fastly executed on cache.
- Cache will store frequently asked query and then if we user requests it, then it will check if it is not stored in cache or not.
- Helped improve latency.



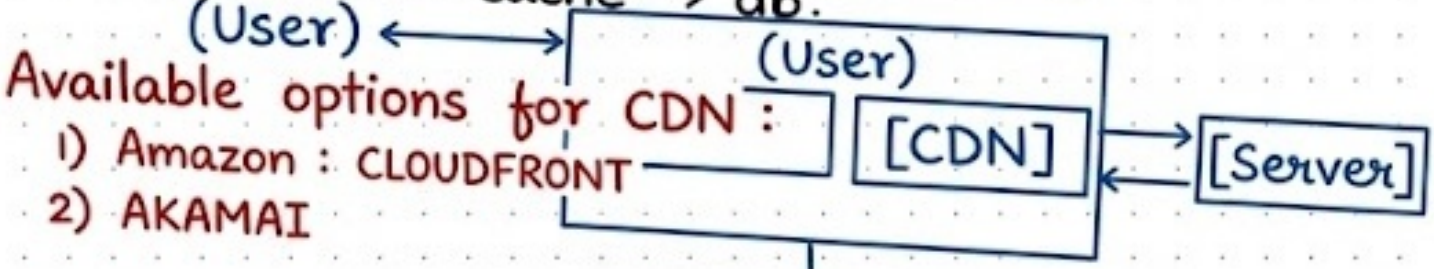
✳ Improved Version : Static content → CDN

- If there are too much static content in our website then it is waste to refresh and bring all data from backend/server.

- Therefore to avoid this we use CDN - Content Delivery Network.

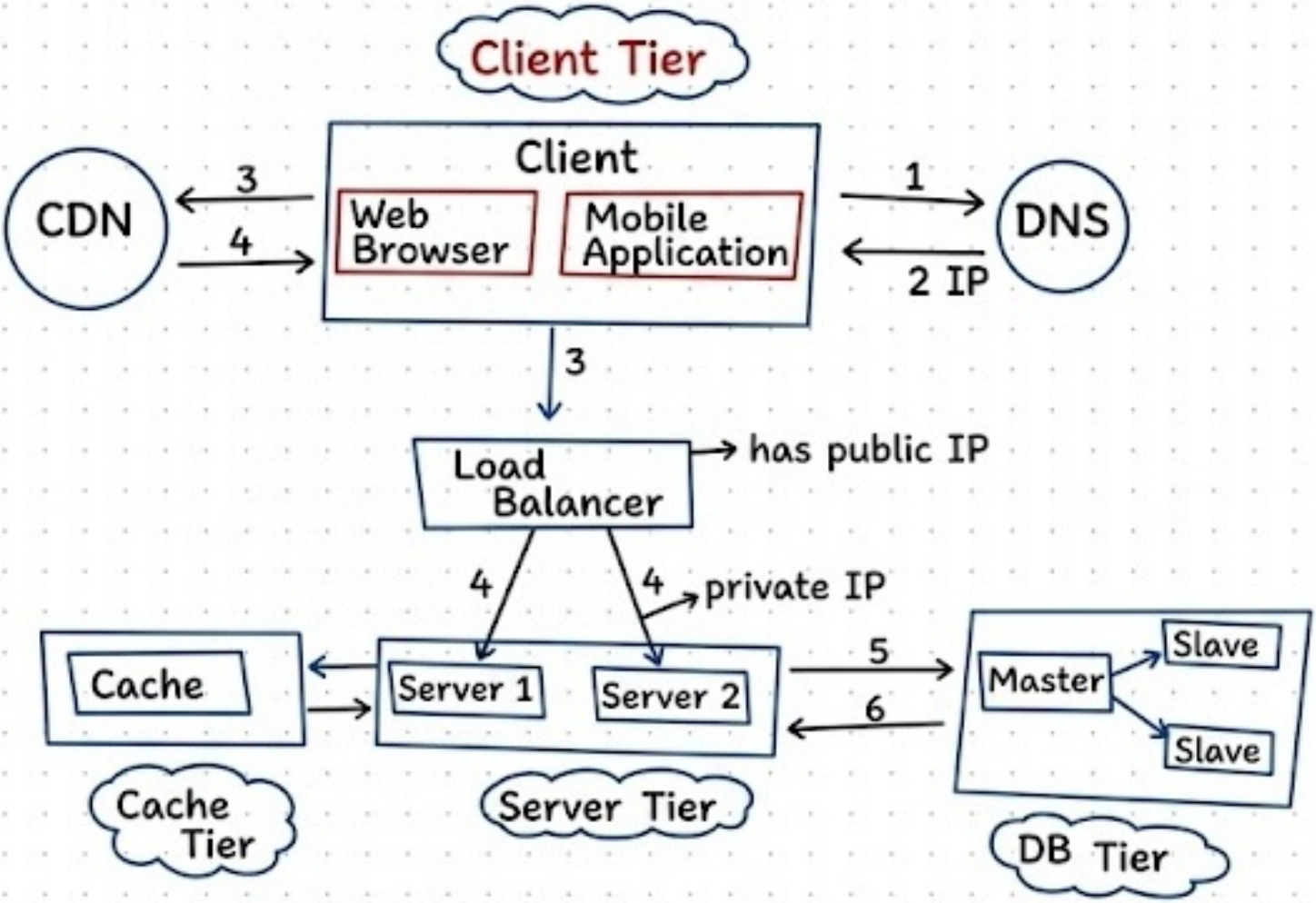
↳ What is CDN ?

- It 's a 3rd party vendor like DNS, CDN acts like proxy of web server
- It's task is to (load) static data without making the process go through load balancer → cache → db.



Available options for CDN :
1) Amazon : CLOUDFRONT
2) AKAMAI

- Geolocation of CDN matters the most
- If Client requests for data, CDN checks all the available server and selects the one which is closer.
- DNS mein ab humare CDN ki IP address store hoga.



Improved Version - Auto Scaling of Servers

Why do we need auto scaling?

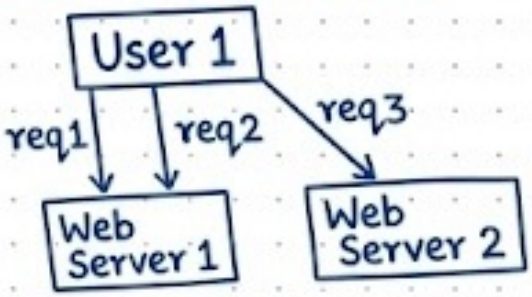
① Agar humare application par less users aa rahe hai toh bohot sare servers and can up rakh kar loss hoga ⇒ depending on traffic of users system/server scale down ho jana chahiye.

② Incase, users bad gaye toh instead of latency and server crash, more server should be available i.e. scale up hona chahiye.

Before understanding auto-scaling we need to understand about Client Sessions.

- Supposer user 1 ne login kiya on web server 1 and uske baad usne kuch request kiya before fulfilling request usne verify karna ki kya user login hai i.e., uska client session active hai.

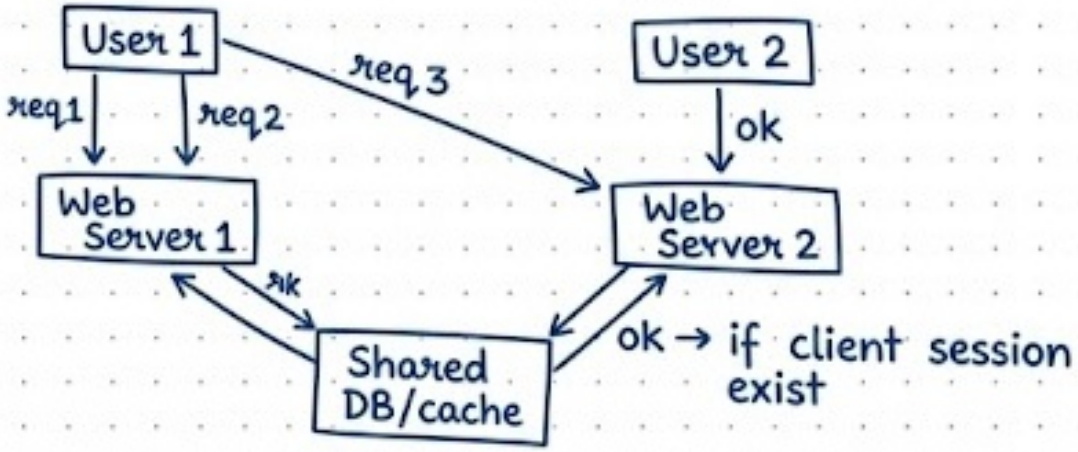
Client Session: User 1



- ab aisa ho sakta hai load balancer next request webserver 2 par bhej de par, uske pass user ki client session ki information hi nhi hogi.
- So server 2, req ka response nhi dega.
- isse kehte hai sticky sessions/ stateful architecture
  ↳ . bekar !!

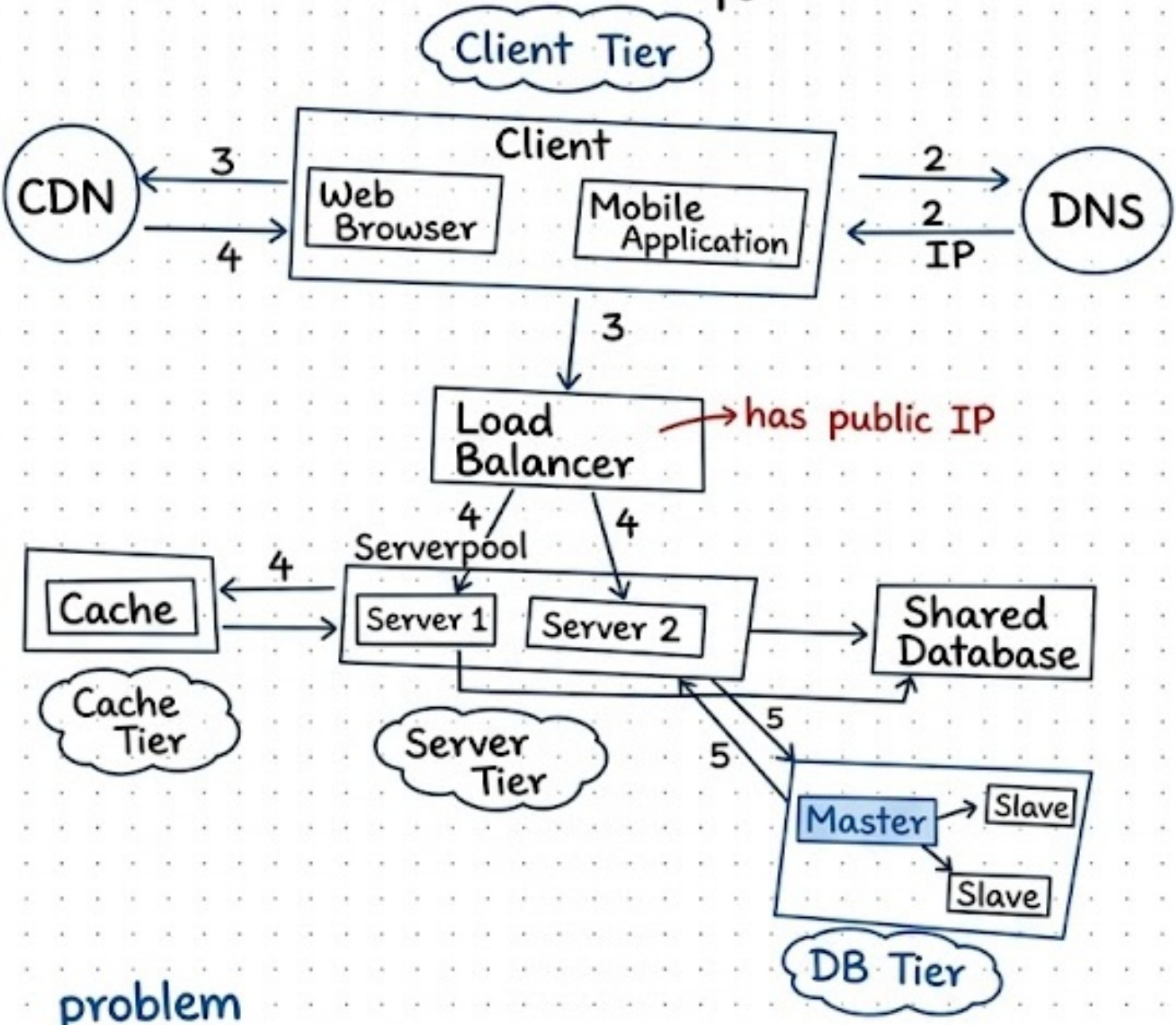
⊗ Solution :

- hum architecture from stateful se stateless le jaye
- kaise? ek separate db introduce krange joh store karega client sessions ko and before webservers responds to any request woh db mein check karega that whether client session exists or not.



- HTTP Protocol stateless hota hai, i.e., it does not require information of client Session.
  ↳ Server ke liye har request unique hoti hai.

- for autoscaling of servers we will use stateless architecture by having shared db for client session. info.



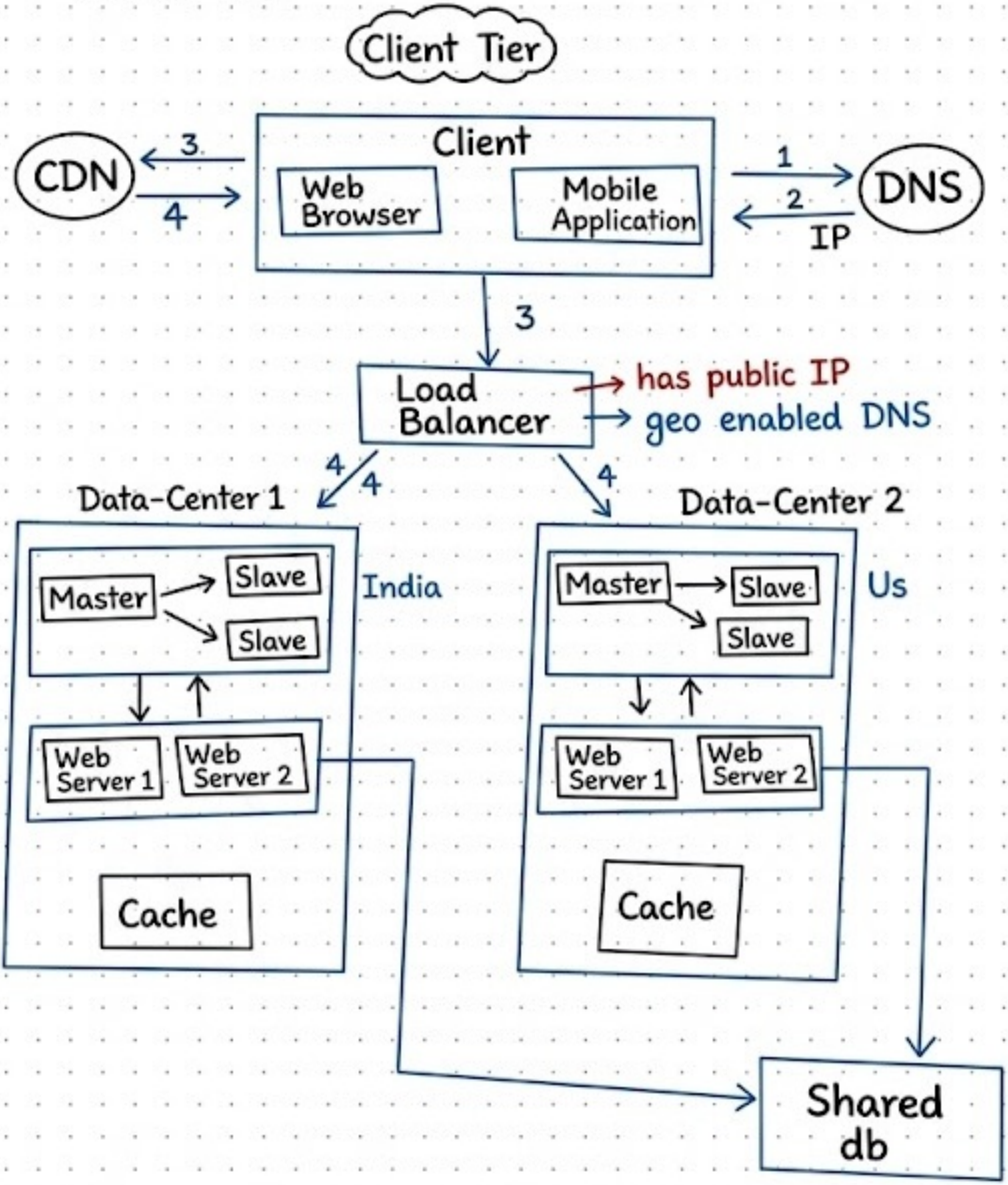
problem

1) yeh website for indian users mast chalegi par agar internasional ke liye slow hogi because uske saari data centers India mein honge.

2) And data packets ko travel krne mein bahut time lagega.

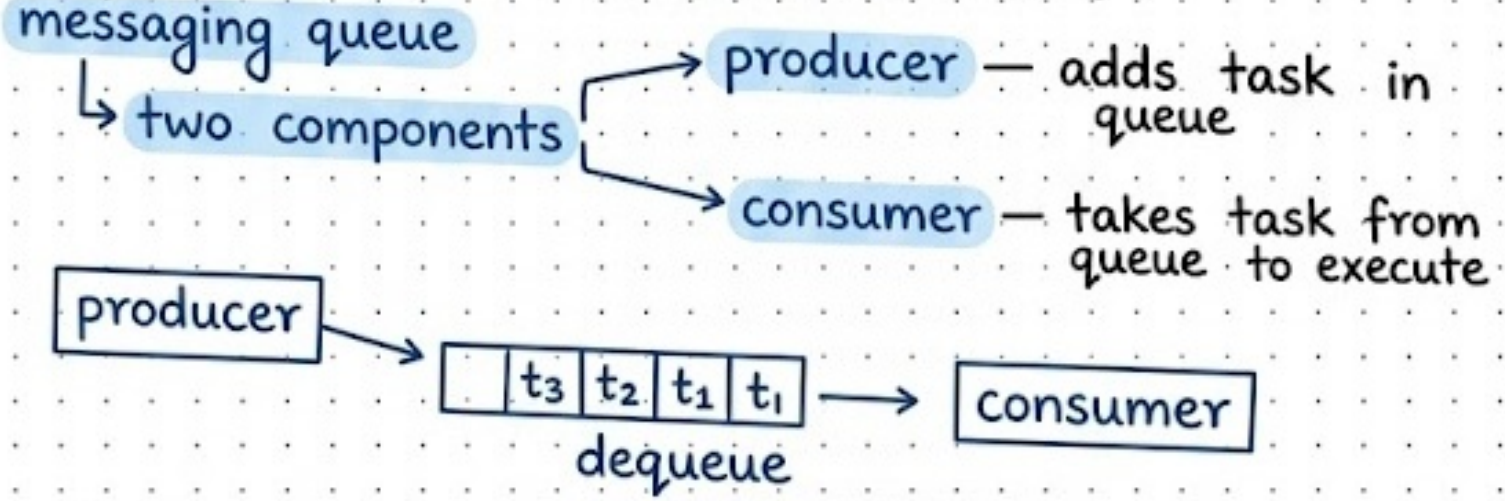
⊗ Improved Version : Multiple Data Center

- Ismein ek hi shared db hota hai jo alag alag geo-locations ke data center se coordinate karta hai.
- Load balancer geo enabled dns par kaam karta hai.



⊕ Improved Version : Messaging Queue

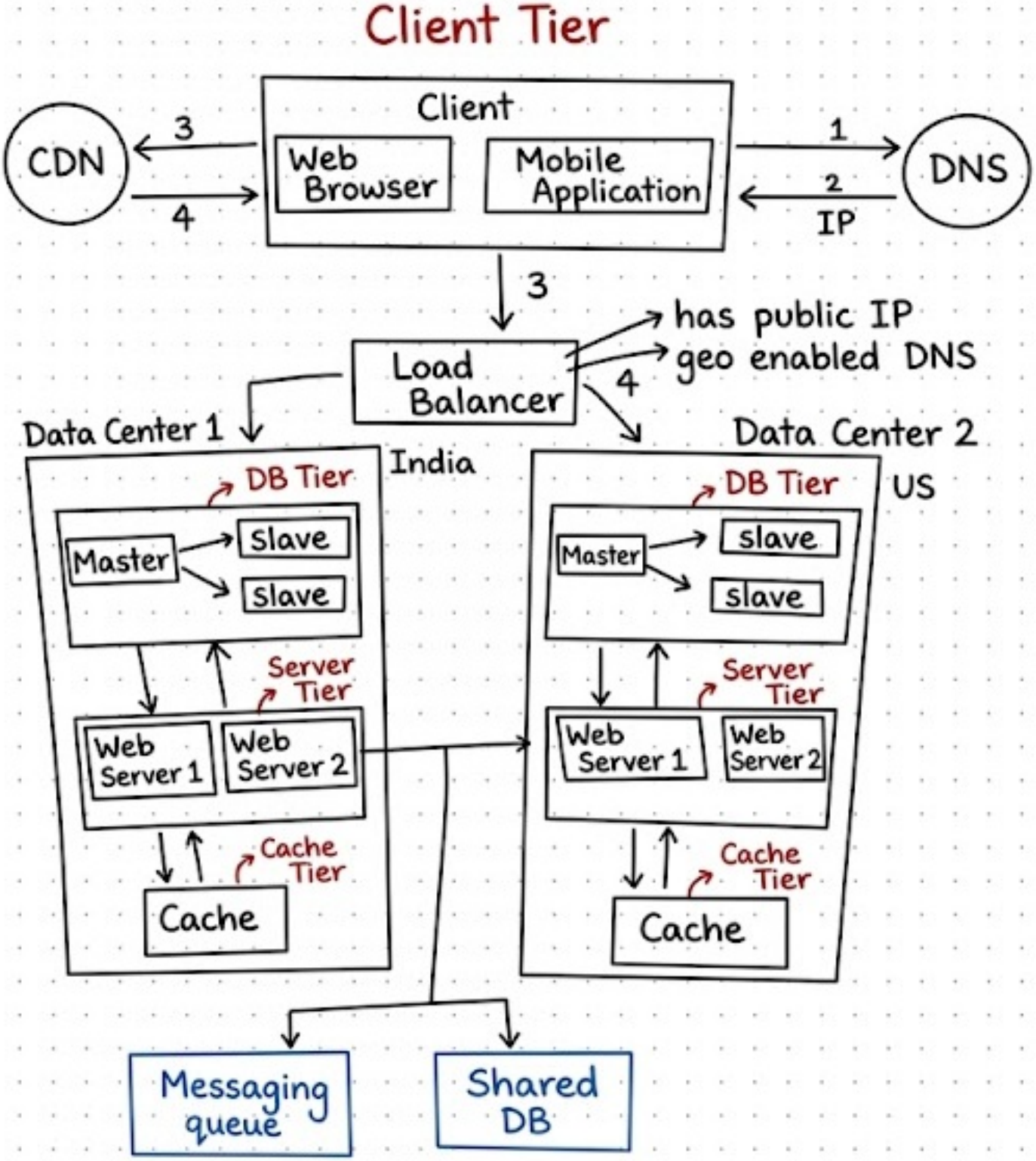
1. Jaise hi humara LB koi req server par bhejta hai toh server busy ho gaya, uss request ko satisfy karne main - aise hi bohot saare kaam agaye toh isse latency increase ho jayegi ⇒ solution : messaging queue.



- Isse hum "publisher-subscriber" model bhi bolte hai.

Types of messaging queue

1) Kafka
2) RabbitMQ


Client Tier
Client
CDN
DNS
Web Browser
Mobile Application
3
4
1
2
IP
3
Load Balancer
has public IP
geo enabled DNS
4
Data Center 1
India
Data Center 2
US
DB Tier
Master
Slave
Slave
Server Tier
Web Server 1
Web Server 2
Cache Tier
Cache
DB Tier
Master
slave
slave
Server Tier
Web Server 1
Web Server 2
Cache Tier
Cache
Messaging queue
Shared DB

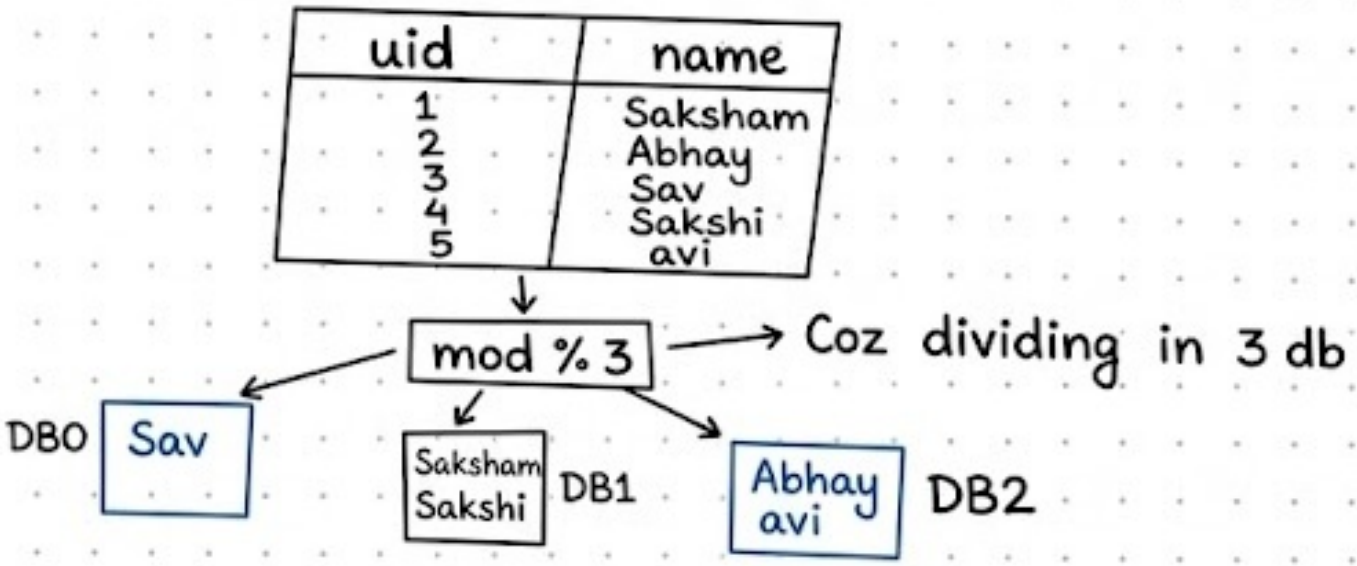
⊛ Final Implementation : DB sharding / partitioning

↳ horizontal scaling

1) Now suppose our db can store 10 million users but we need more so we can increase ram, still not enough.
2) then we consider horizontal scaling, i.e. dividing data into multiple database

Ex of SQL db

we divide data based on mod 3



- mod % 3 on uid → known as sharding key
- db0, db1, db2 → known as shards
- Now if user wants uid=12 it will just mod % 3 & search in that shard/db
- Now what if all shards are full we add one more shard/db but we need to divide data and the process is known as resharding
- Resharding is quite resource extensive, time taking process and can be achieved by data de-normalization